

Marketplace Analytics & Personalization for Service Platforms

Introduction

This case study compilation highlights the design and implementation of two key data products built for a service marketplace platform that enables users to book appointments via a mobile application. Our goal was to enhance user engagement and platform integrity using scalable, low-latency data solutions. These case studies focus on two core initiatives:

1. **Ranking Optimization for Platform Loyalty** — discouraging off-platform bookings via a merit-based service ranking system.
2. **Real-Time Personalization** — delivering personalized recommendations in-session using event streaming and behavioral models.

Both solutions prioritize simplicity, scalability, and business impact, using tools such as Apache Spark, Redshift, Airflow, and Amazon Kinesis.

Case Study 1: Merit-Based Ranking to Encourage Platform Loyalty

Problem

Some service providers were completing bookings outside the app, reducing platform visibility and data coverage. To counter this, we introduced a **merit-based ranking system** that rewards exclusive usage by improving a provider's in-app ranking. This system discourages off-platform activity by reducing visibility for providers that frequently take bookings outside the app.

Technical Solution

- ◆ Scalable Reindexing with PySpark on Amazon EMR
 - **Why Spark?** Needed to efficiently process millions of service-provider records.

- **Parallel Processing:** Batched queries and transformations run across distributed nodes.
- **Transformations:** Included deduplication, behavioral tagging, and metadata normalization.

◆ Direct Indexing into ElasticSearch

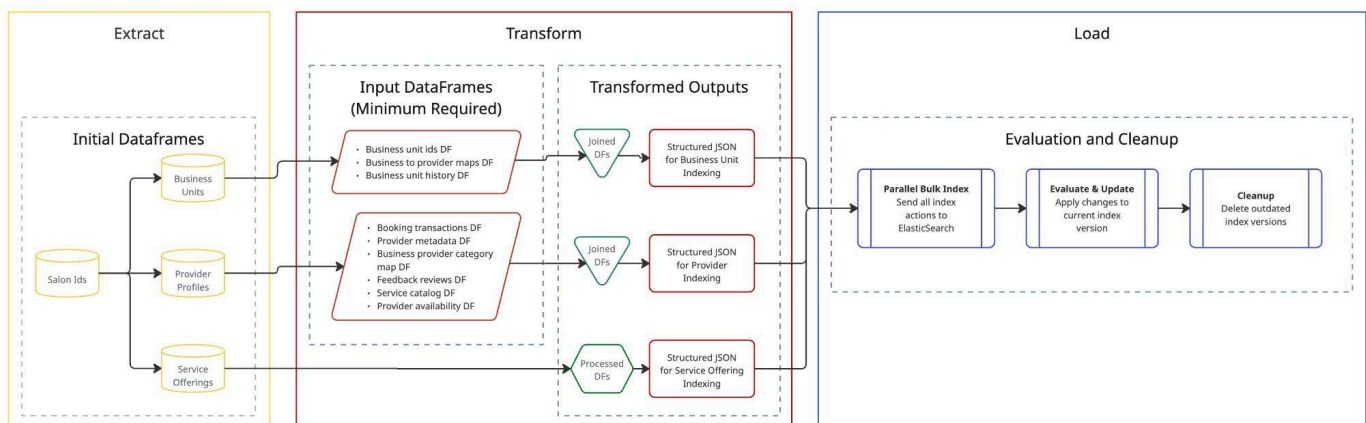
- **Integration:** Used `elasticsearch-hadoop` for direct Spark-to-ElasticSearch writes.
- **Bulk Indexing:** Implemented using Python's `parallel_bulk`, reducing latency and retries.
- **Work Availability Check:** During data transformation, we calculate gaps in each provider's calendar to infer unaccounted-for work hours. If a provider has consistent unavailability but few or no bookings through the platform, it may indicate external booking behavior. This signal feeds into ranking logic during indexing.

◆ Workflow Orchestration via Apache Airflow

- **DAG Structure:** Segmented into extract → transform → index tasks.
- **Parallelism:** Multiple Spark jobs orchestrated through Airflow and indexing threads run simultaneously.
- **Monitoring:** Airflow UI allowed easy debugging, log tracking, and job health checks.

◆ Indexing Strategy

- **Chunking:** Data split into parallelizable units for efficiency.
- **Retry Logic:** Failed bulk operations logged and retried without re-running full pipeline.
- **Batch Control:** Dynamically tuned based on workload and resource utilization.





Business Outcome

- ◆ Created an Incentive Loop

The merit-based ranking system ties a provider's in-app visibility directly to their platform engagement. Providers who consistently book through the app are ranked higher in search results, which leads to more exposure and bookings. This self-reinforcing loop encourages continued platform loyalty while discouraging off-platform scheduling.

- ◆ Improved Data Completeness and Marketplace Integrity

By penalizing providers who appear unavailable but have no bookings within the app (a proxy for off-platform behavior), we closed key data gaps. This improved the accuracy of availability data and ensured the platform reflects true service supply, helping users trust that what they see is what's actually available.

- ◆ Operational Efficiency at Scale

The new reindexing pipeline is fully automated, scalable, and fault-tolerant. It processes millions of provider records using Spark on EMR, orchestrated through Airflow, and directly indexed into Elasticsearch. This eliminated the need for manual reindexing efforts and reduced latency, enabling the platform to maintain fresh, behavior-based rankings in near real time.

- ◆ Strategic Advantage

Beyond operational improvements, the ranking logic explicitly incentivizes providers to book exclusively through the platform by boosting their visibility and attracting more customers. Meanwhile, users are implicitly rewarded by having their favorite service providers accessible only within the app, ensuring a consistent, reliable booking experience. This dual-sided effect strengthens network effects, solidifies the platform as the default booking channel, and drives sustainable marketplace growth.

Case Study 2: Real-Time Personalization Using Behavioral Modeling

Problem

The client's app offered limited personalization, showing static or generic recommendations that didn't reflect individual behavior or contextual interest. Additionally, it missed opportunities to **cross-sell related services** based on prior bookings or behavioral patterns.

We needed to:

1. Show **real-time personalized recommendations** based on user behavior.
2. Surface **relevant cross-sell suggestions** at key decision points (e.g., after booking a service).

Dual Solution Strategy

We implemented two complementary solutions:

- A **real-time scoring pipeline** based on user actions using behavioral modeling.
- A **cross-sell recommendation engine** using market basket analysis with 563 custom and learned rules.

◆ Part 1: Real-Time Personalization via Event Streaming and Scoring

Real-Time Event Streaming with Amazon Kinesis

- Tracked events: searches, search clicks, booking clicks, confirmation views, and bookings.
- Events were streamed via Kinesis to the backend for continuous processing.

Scoring with Logistic Regression in Redshift

- Model trained on previous interactions.

- Signals weighted based on the booking likelihood of the signal.
- Implemented directly in Redshift for zero-infrastructure scoring.

Performance

- **86% accuracy, 87% precision, MRR: 0.74**
 - Accurately identifies services users are most (and least) interested in
 - Easy to update with new signals or recency-based decay logic
-

◆ Part 2: Cross-Sell Recommendations with Market Basket + Custom Rules

Goal

Recommend add-on services based on a user's current or prior bookings.

Methodology

- **4.2M transactions, 1.2M users, 481 services**, 13 months of data
- Used **association rule mining** with:
 - Min support: 0.81%
 - Confidence threshold: 7.9%
- For uncovered services, custom rules were created based on popularity and similarity

Results

- **563 rules total**: 127 statistically mined, 436 custom

Model Evaluation

Metric	Score
Hit Rate	36.6%
Precision@5	13.2%

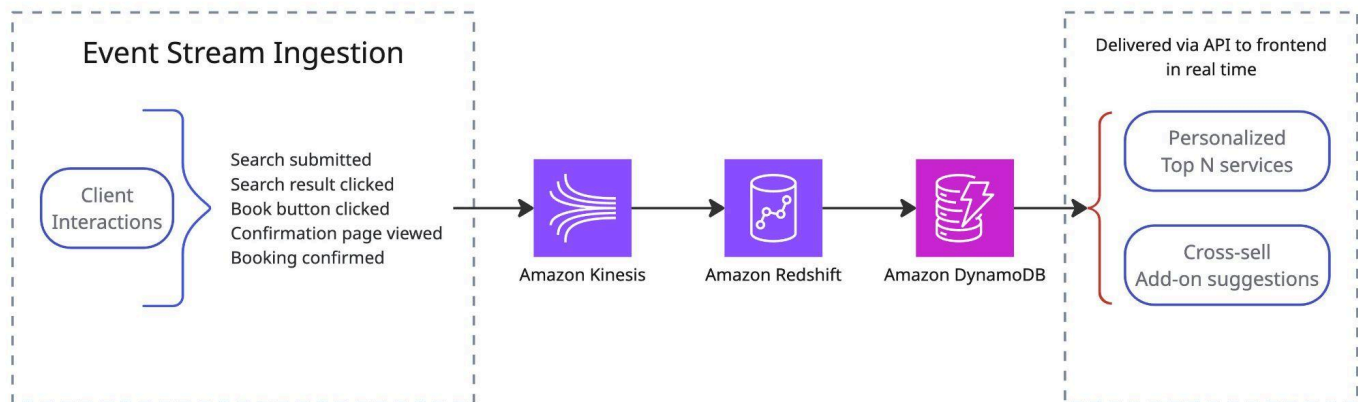
Recall@5 35.6%

MRR 25.5%

Coverage 4.6%

Tradeoff Analysis

- Popular services drive higher success but reduce variety
- A balance between **safe** vs. **diverse** recommendations is needed
- Future: Explore hybrid methods with collaborative filtering



Business Impact

◆ Real-Time, Behavior-Aware Personalization

The recommendation system now adapts to user behavior as it happens. Actions like searches, booking attempts, or confirmation views immediately influence the personalized services shown to each user. This creates a dynamic, responsive experience that reflects current intent—not just historical behavior, leading to higher engagement and relevance.

◆ Contextual Cross-Sell at the Right Moment

Intelligent cross-sell recommendations are shown based on what the user has just booked or browsed. These rules are derived from real-world patterns. The system ensures every service has at least one relevant suggestion, increasing basket size without overwhelming the user.

- ◆ No Complex ML Infrastructure Required

Both the personalization and cross-sell logic are implemented entirely within Redshift. This avoids the need for separate inference servers or online ML endpoints. Real-time behavioral inputs are streamed via Amazon Kinesis, and logistic scoring or rule evaluation is done in-place. This approach drastically reduces operational complexity and deployment time.

- ◆ Lift in Conversion and Booking Value

By combining relevance (personalization) with timely nudges (cross-sell), users are more likely to book quickly and add additional services. Early metrics showed increased click-through rates on personalized modules and strong engagement with cross-sell cards.

- ◆ Scalable Foundation for Experimentation

The architecture supports future enhancements such as recency-based weighting, A/B testing of recommendation logic, or layering collaborative filtering. As user activity grows, the platform remains equipped to scale personalization without introducing infrastructure overhead.